COSC 120: Introduction to Python Programming

In-Class Midterm 2 Examination – Version 3

Prof. Jordan C. Hanson

Name: _____ **Student ID:** [ ]

Instructions: Choose the best answer for each question. Fill in the circle next to exactly one answer, A–D. Each question is worth 4 points.

Answer Sheet

1 ◯A ◯B ◯C ◯D	**2** ◯A ◯B ◯C ◯D	**3** ◯A ◯B ◯C ◯D	**4** ◯A ◯B ◯C ◯D	**5** ◯A ◯B ◯C ◯D
6 ◯A ◯B ◯C ◯D	**7** ◯A ◯B ◯C ◯D	**8** ◯A ◯B ◯C ◯D	**9** ◯A ◯B ◯C ◯D	**10** ◯A ◯B ◯C ◯D
11 ◯A ◯B ◯C ◯D	**12** ◯A ◯B ◯C ◯D	**13** ◯A ◯B ◯C ◯D	**14** ◯A ◯B ◯C ◯D	**15** ◯A ◯B ◯C ◯D
16 ◯A ◯B ◯C ◯D	**17** ◯A ◯B ◯C ◯D	**18** ◯A ◯B ◯C ◯D	**19** ◯A ◯B ◯C ◯D	**20** ◯A ◯B ◯C ◯D
21 ◯A ◯B ◯C ◯D	**22** ◯A ◯B ◯C ◯D	**23** ◯A ◯B ◯C ◯D	**24** ◯A ◯B ◯C ◯D	**25** ◯A ◯B ◯C ◯D

Dictionaries and Comprehensions

1. Consider the dictionary

```
students = {
    2601001: "Ana",
    2601002: "Ben",
    2601003: "Chloe"
}
```

What is produced by

```
students[2601002]
```

◯ **A.** Ana

◯ **B.** Ben

◯ **C.** 2601002

◯ **D.** An error occurs

2. Suppose the key `2601999` is not in `students`. What is returned by

```
students.get(2601999)
```

◯ **A.** 0

◯ **B.** False

◯ **C.** None

◯ **D.** The program always stops with an error

3. Consider

```
names = ["Ana", "Ben", "Chloe",
         "Diego", "Elena"]
first_id = 2601001
students = {
    first_id + i: name
    for i, name in enumerate(names)
}
```

Which key is associated with `"Diego"`?

◯ **A.** 2601001

◯ **B.** 2601002

◯ **C.** 2601003

◯ **D.** 2601004

4. Which statement best describes the difference between these two expressions?

```
[first_id + i
 for i in range(len(names))]

{first_id + i: name
 for i, name in enumerate(names)}
```

◯ **A.** The first creates a list of IDs; the second creates ID–name pairs in a dictionary

◯ **B.** Both create dictionaries

◯ **C.** The first creates a dictionary; the second creates a list

◯ **D.** Both create exactly the same object

5. In our lookup-timing exercise, why did we use

```
time.perf_counter()
```

◯ **A.** To sort the student records

◯ **B.** To measure short execution times with a high-resolution timer

◯ **C.** To count how many dictionary keys exist

◯ **D.** To pause execution for a fixed number of seconds

6. A student ID is near the end of a nested list containing N records. Which statement best describes our timing experiment as N becomes large?

◯ **A.** Both nested-list and dictionary lookup times must grow in exactly the same way

◯ **B.** Dictionary lookup becomes impossible once $N > 1000$

◯ **C.** Nested-list lookup generally grows with N, while dictionary lookup changes much less

◯ **D.** Nested-list lookup becomes constant-time because the list is nested

Classes and Objects

7. In a class definition, what is the main purpose of

```
def __init__(self, ...):
```

◯ **A.** To initialize a new object's attributes

◯ **B.** To delete an object

◯ **C.** To import another module

◯ **D.** To begin a recursive call

8. Inside an instance method, what does `self` refer to?

◯ **A.** The class file on disk

◯ **B.** Every object that has ever been created

◯ **C.** The Python interpreter

◯ **D.** The particular object on which the method was called

9. Consider the complete class fragment

```
class Particle2D:
    def __init__(self, x, y, vx, vy):
        self.x = x
        self.y = y
        self.vx = vx
        self.vy = vy

    def update(self, dt):
        self.x += self.vx * dt
        self.y += self.vy * dt

p = Particle2D(1, 2, 3, -1)
p.update(0.5)
print(p.x, p.vx)
```

What is printed?

◯ **A.** `2.5 3`

◯ **B.** `2.5 1.5`

◯ **C.** `1 3`

◯ **D.** `4 -1`

10. Using the same `Particle2D` class, consider

```
p1 = Particle2D(0, 0, 2, 1)
p2 = Particle2D(0, 0, -1, 3)

p1.update(1.0)
print(p2.x, p2.y)
```

What is printed, and why?

◯ **A.** `2 1`, because all instances share the same position

◯ **B.** `-1 3`, because creating `p2` automatically updates it

◯ **C.** `0 0`, because `update()` was called only on `p1`

◯ **D.** An error occurs because two instances cannot exist at once

11. A student writes the following method inside `Particle2D`:

```
def speed(self):
    return math.sqrt(vx**2 + vy**2)
```

Why will this method fail when called as `p.speed()`?

◯ **A.** The method should use `self.vx` and `self.vy`

◯ **B.** `math.sqrt()` cannot be used inside a class

◯ **C.** Instance methods cannot return values

◯ **D.** `speed()` must change `x` and `y`

12. Let

```
z1 = ComplexNumber(3, 2)
z2 = ComplexNumber(5, -4)
z3 = z1 + z2
```

If `__add__()` correctly implements complex-number addition, and `z3.real` and `z3.imag` produce the real and imaginary parts of `z3`, what are `z3.real` and `z3.imag`?

◯ **A.** `15, -8`

◯ **B.** `-2, 6`

◯ **C.** `8, 6`

◯ **D.** `8, -2`

13. Consider the method

```
def __mul__(self, other):
    real = (self.real * other.real
            - self.imag * other.imag)
    imag = (self.real * other.imag
            + self.imag * other.real)
    return ComplexNumber(real, imag)
```

Which statement best describes what happens when Python evaluates `z3 = z1 * z2`?

◯ **A.** A new `ComplexNumber` object is created and assigned to `z3`

◯ **B.** `z1` is permanently changed into the product

◯ **C.** The method returns a two-element list

◯ **D.** Python ignores `__mul__()` because `z1` and `z2` are objects

Recursion and Binary Search

14. Which statement best defines a **base case** in a recursive function?

◯ **A.** The first line of the function

◯ **B.** The case that makes the largest recursive call

◯ **C.** A stopping case that returns without making another recursive call

◯ **D.** A loop that always runs at least once

15. Consider the recursive pattern from class:

```
def all_positive(values):
    if len(values) == 0:
        return True
    x = values.pop()
    if x <= 0:
        return False
    return all_positive(values)
```

What is returned for

```
all_positive([6, 2, -9, 4])
```

◯ **A.** False

◯ **B.** True

◯ **C.** -9

◯ **D.** None

16. Consider

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n - 1)
```

When evaluating `fact(4)`, which sequence of function calls is made before the base case returns?

◯ **A.** `fact(4), fact(3), fact(2), fact(1)`

◯ **B.** `fact(4), fact(5), fact(6), ...`

◯ **C.** `fact(4)` only

◯ **D.** `fact(1), fact(2), fact(3), fact(4)`

17. A student writes

```
def fact(n):
    if n <= 1:
        return 1
    return n * fact(n)
```

Why does this implementation fail for `fact(5)`?

◯ **A.** The base case should return 0

◯ **B.** The recursive call uses the same `n`, so it never moves toward the base case

◯ **C.** Multiplication is not allowed in a recursive function

◯ **D.** Recursive functions must contain a `for` loop

18. Consider this part of recursive binary search:

```
mid = (low + high) // 2

if values[mid] == target:
    return mid
if target < values[mid]:
    return binary_search(
        values, target, low, mid - 1)
return binary_search(
    values, target, mid + 1, high)
```

For

```
values = [3, 7, 11, 18, 23, 29, 34]
target = 29
low = 0
high = 6
```

the first midpoint is index 3, whose value is 18. What recursive call is made next?

◯ **A.** `binary_search(values, 29, 0, 2)`

◯ **B.** `binary_search(values, 29, 4, 6)`

◯ **C.** `binary_search(values, 29, 0, 6)`

◯ **D.** The function immediately returns `-1`

19. In our recursive binary-search function, what does

```
if low > high:
    return -1
```

mean?

◯ **A.** There are no indices left to search, so the target is absent

◯ **B.** The target must be at index 0

◯ **C.** The list should be reversed

◯ **D.** The target has been found at `mid`

20. Why is binary search efficient on a large sorted list?

◯ **A.** It always examines every element exactly once

◯ **B.** It converts the list into a dictionary first

◯ **C.** It works only because recursive functions are always constant-time

◯ **D.** Each decision discards about half of the remaining search region

Inheritance and Vector2D

21. Consider the class method

```
def __add__(self, other):
    return Vector2D(
        self.x + other.x,
        self.y + other.y)
```

and the code

```
v1 = Vector2D(2, 5)
v2 = Vector2D(-1, 3)
v3 = v1 + v2
```

Which statement is correct?

◯ **A.** `v3` is a `Vector2D` with components `(1, 8)`

◯ **B.** `v3` is the integer `9`

◯ **C.** `v1` is changed to `(-1, 3)`

◯ **D.** Python cannot use `+` with objects

22. Suppose `ComplexNumber` inherits from `Vector2D` and overrides only `__add__()`:

```
class ComplexNumber(Vector2D):
    def __add__(self, other):
        return ComplexNumber(
            self.x + other.x,
            self.y + other.y)
```

If `Vector2D` already defines `magnitude()`, which methods are used by the following expressions?

```
z1.magnitude()
z1 + z2
```

◯ **A.** Both use methods defined only in `Vector2D`

◯ **B.** `magnitude()` is inherited from `Vector2D`; addition uses the overridden method in `ComplexNumber`

◯ **C.** Both require methods to be rewritten inside `ComplexNumber`

◯ **D.** `magnitude()` uses `ComplexNumber`; addition uses `Vector2D`

23. Consider

```
class ComplexNumber(Vector2D):
    ...
```

Which statement is correct?

◯ **A.** `ComplexNumber` is a child class that inherits from `Vector2D`

◯ **B.** `Vector2D` inherits from `ComplexNumber`

◯ **C.** The two classes are unrelated

◯ **D.** Python combines the two classes into a dictionary

24. In

```
def __init__(self, real, imag):
    super().__init__(real, imag)
```

what does `super().__init__()` do?

◯ **A.** It creates an unrelated `Vector2D` object and discards it

◯ **B.** It calls the recursive base case

◯ **C.** It overrides every method in the parent class

◯ **D.** It reuses the parent class constructor to initialize the inherited components

25. Suppose the parent and child classes contain

```
class Vector2D:
    def __add__(self, other):
        return Vector2D(
            self.x + other.x,
            self.y + other.y)

class ComplexNumber(Vector2D):
    def __add__(self, other):
        return ComplexNumber(
            self.x + other.x,
            self.y + other.y)
```

For two `ComplexNumber` objects `z1` and `z2`, what is the difference between

```
a = Vector2D.__add__(z1, z2)
b = z1 + z2
```

◯ **A.** Both `a` and `b` are `Vector2D` objects

◯ **B.** `a` is a `Vector2D`; `b` is a `ComplexNumber`

◯ **C.** `a` is a `ComplexNumber`; `b` is a `Vector2D`

◯ **D.** Both statements produce an error

End of Exam

Check that exactly one answer is clearly marked for each question.